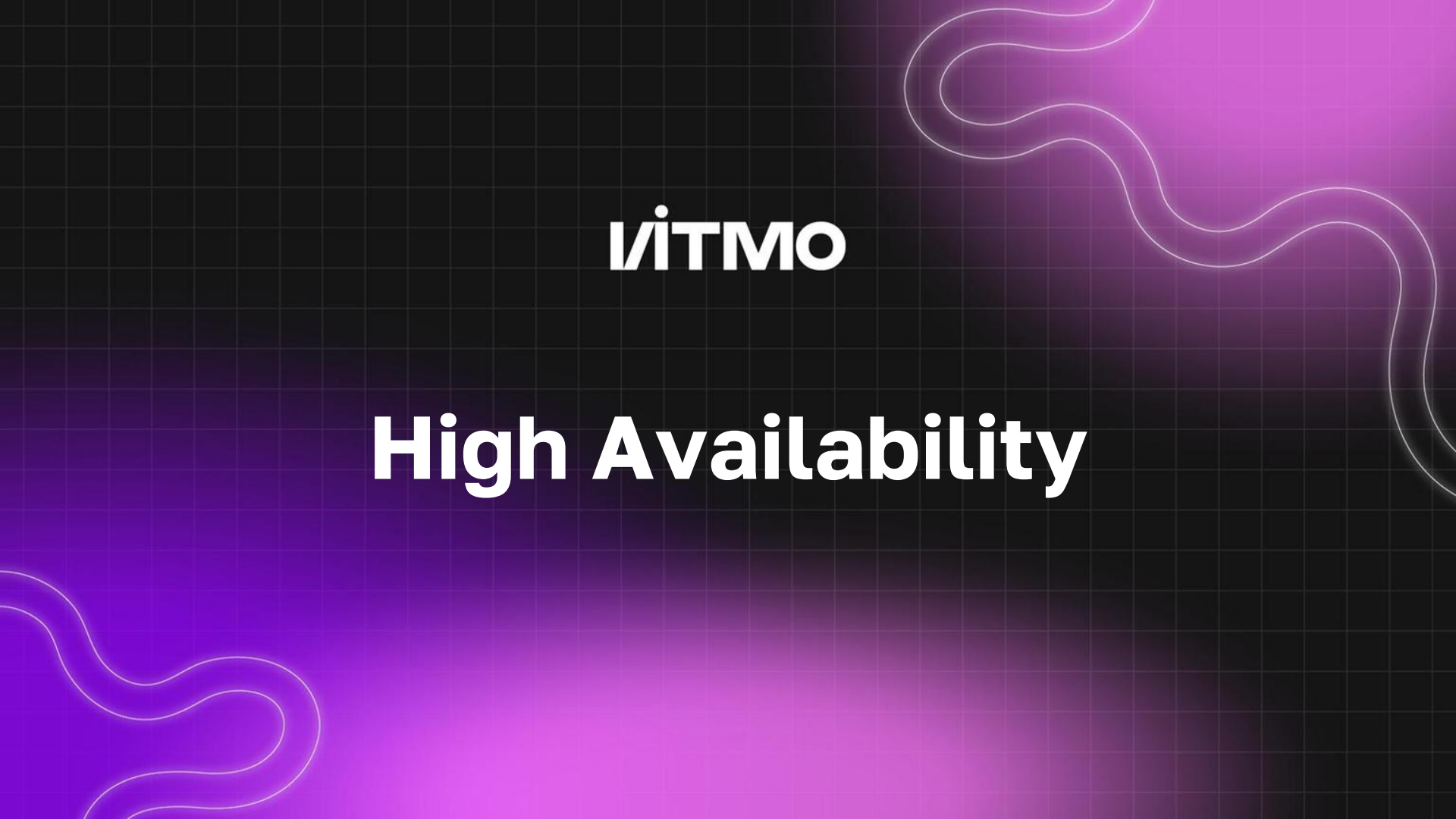




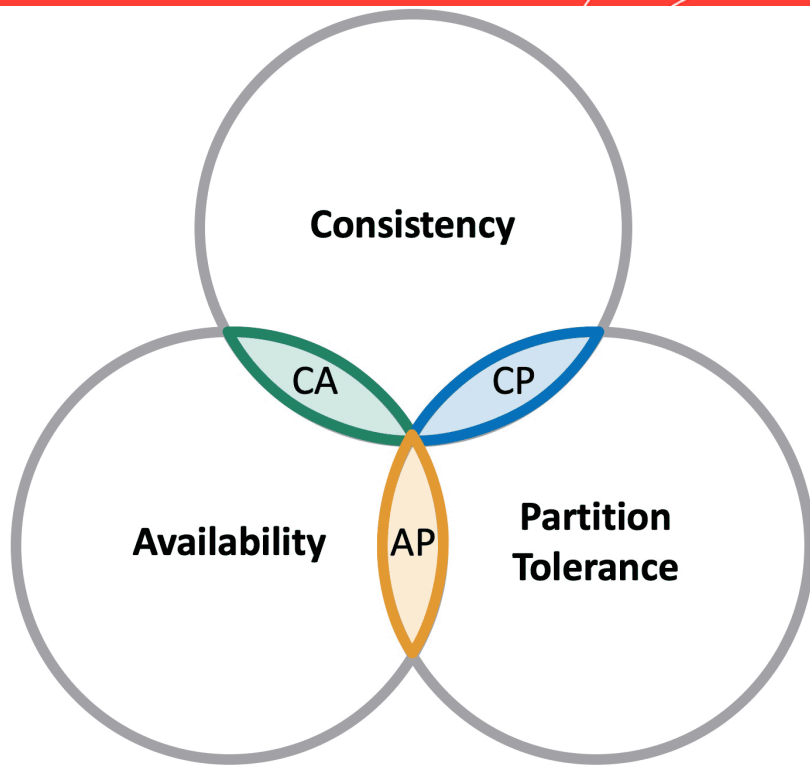
IITMO

High Availability



- Архитектурные решения
- Программные архитектуры и сервисы
- Дублирование данных
- Дублирование питания и систем связи
- Дублирование оборудования
- Дублирование сервисов (кластеризация)
- Обеспечение условий эксплуатации

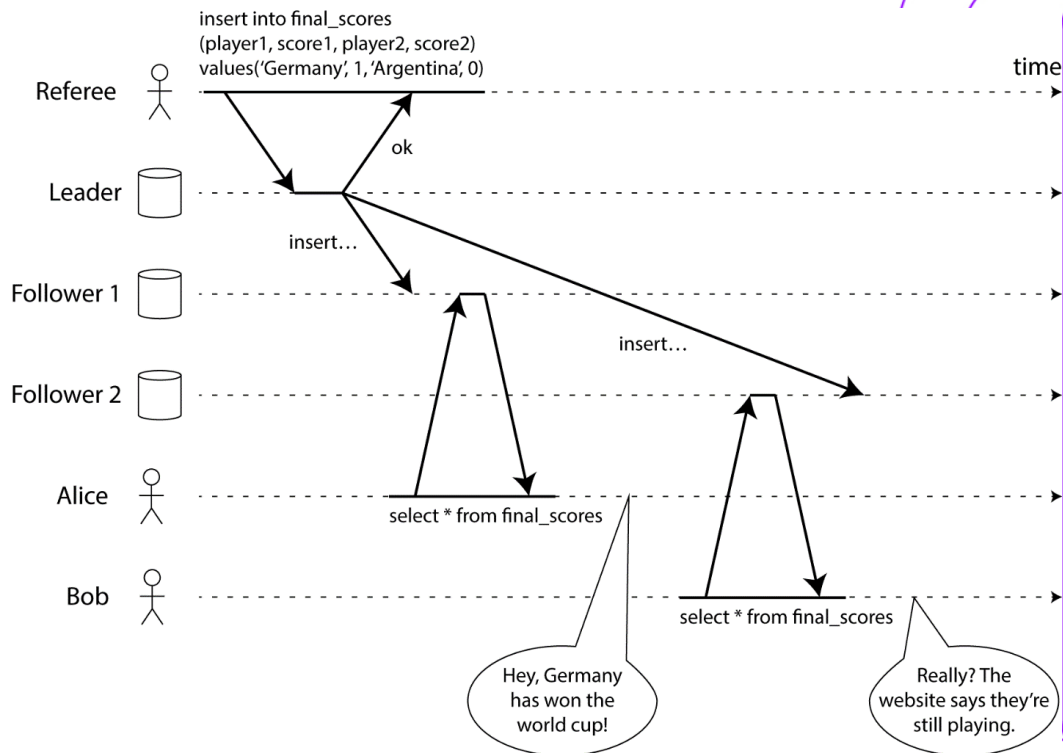
CAP Теорема



Теорема, утверждающая, что распределенная система данных может гарантировать одновременно только 2 из 3 свойств:

- **C (Consistency)** - Согласованность Все узлы видят одни и те же данные в один момент времени. Любое чтение возвращает результат последней записи
- **A (Availability)** - Доступность Каждый запрос получает ответ (успешный или ошибку), даже если некоторые узлы недоступны
- **P (Partition Tolerance)** - Устойчивость к разделению Система продолжает работать, даже если связь между узлами нарушена (network partition)

Что не так с определениями?



CAP - это про поведение во время сбоя, а не в нормальном режиме

Consistency в CAP фактически означает линейризуемость (и ее действительно трудно достичь) + какой тип?

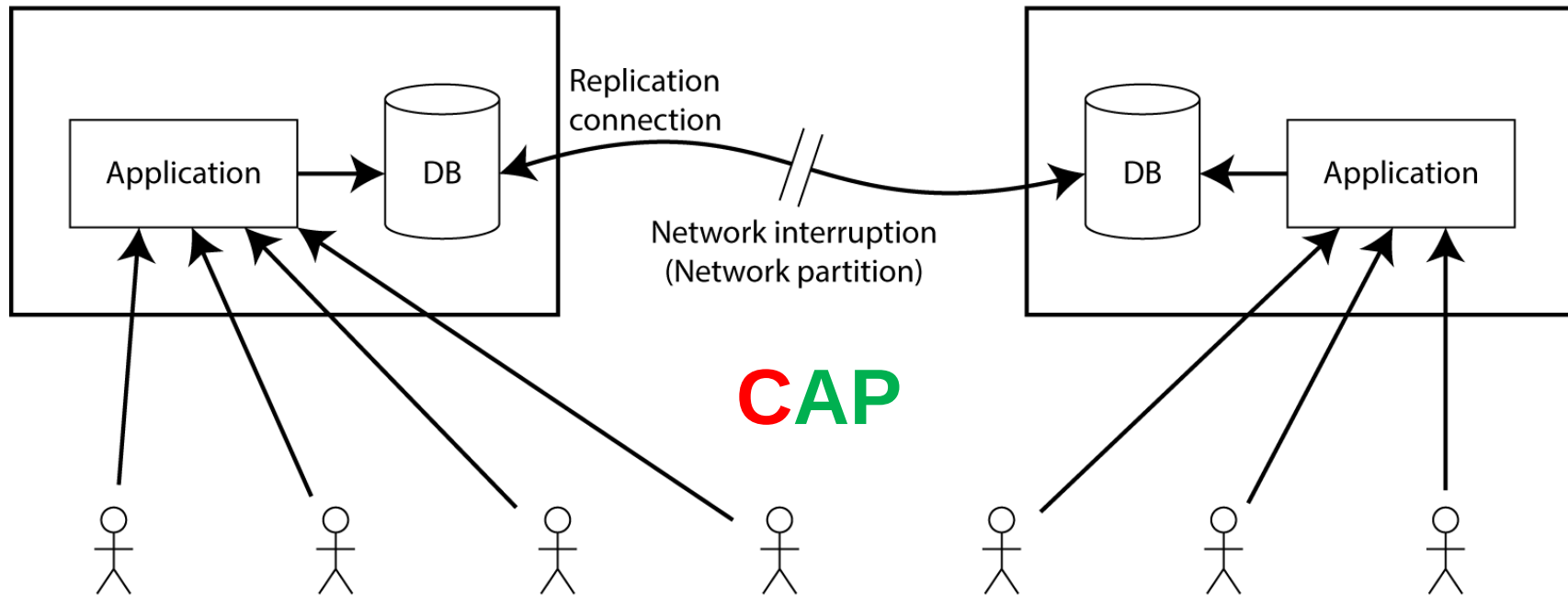
Availability в CAP, исходя из определения имеет две серьёзные проблемы. Первая – нет понятия частичной доступности, или какой-то её степени (проценты например), а есть только полная доступность. Вторая проблема – неограниченное время ответа на запросы, т.е. даже если система отвечает час, она всё ещё доступна

Partit... whatever.

САР и БД

Datacenter 1

Datacenter 2



Clients

Балансировка нагрузки

- Статическая
- Динамическая

- *Round robin* (все по порядку)
- *Least connections* (по использованию)

Зачем?

- Достигаем оптимальную утилизацию ресурсов
- Максимизируем пропускную способность
- Минимизируем время отклика
- Избегаем оверлоада и падений

Удобно администрировать/
управлять, добавлять/удалять
узлы, разделять по типу трафика
и т.д.

Load Balancing



L7 (Application Layer)

- HTTP заголовки, куки, URL
 - SSL termination
 - Роутинг на основе контента
- Примеры: HAProxy, Nginx, Traefik

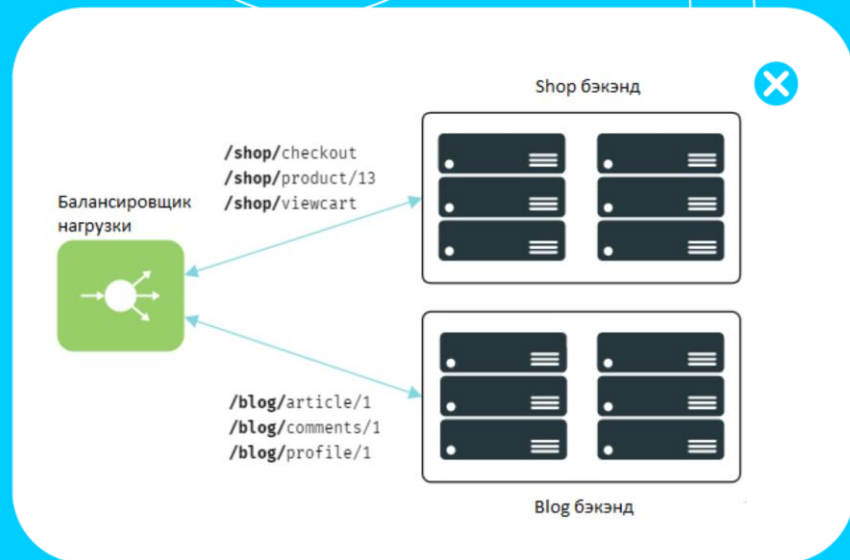
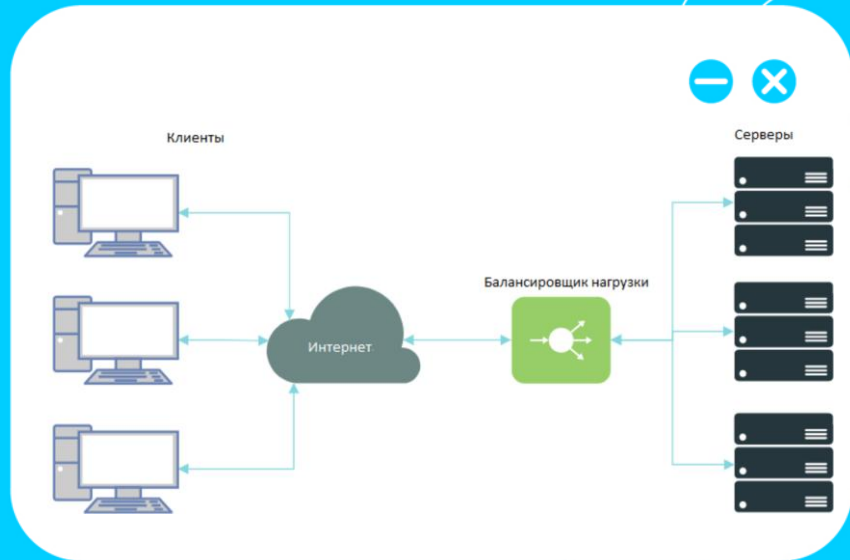
L4 (Transport Layer)

- TCP/UDP port, IP address
 - Быстрее, меньше overhead
- Примеры: HAProxy (TCP mode), iptables

L3 (Network Layer)

- IP routing, ECMP
- Примеры: BGP routing, hardware LB

Примеры





HA PROXY



- Open-source балансировщик нагрузки и прокси-сервер
- Поддержка TCP (Layer 4) и HTTP (Layer 7)
- Высокая производительность: 100k+ соединений/сек

- + Легкий, быстрый
- + Встроенный дашборд
- + Мгновенный reload
- - Не умеет в autodiscovery

Health Checks:

- TCP проверки доступности
- HTTP endpoint мониторинг
- Custom scripts

Load Balancing алгоритмы:

- roundrobin
- leastconn
- source (sticky sessions)
- uri

Failover:

- Автоматическое исключение упавших нод
- Graceful shutdown
- Backup серверы

HAProxy

```
defaults
    mode http
    timeout connect 5s
    timeout client 50s
    timeout server 50s
    timeout check 5s

# =====
# HAProxy Statistics
# =====

listen stats
    mode http
    bind *:7000
    stats enable
    stats uri /

# =====
# Web Application
# =====

listen web_app
    bind *:80
    balance roundrobin
    option httpchk GET /health
    http-check expect status 200
    default-server inter 3s fall 3 rise 2
    server web1 10.0.1.10:8080 check
    server web2 10.0.1.11:8080 check
    server web3 10.0.1.12:8080 check backup
```

ИТМО

/etc/haproxy/haproxy.cfg



- global # Глобальные параметры процесса
- defaults # Значения по умолчанию
- frontend # Точки входа (listeners)
- backend # Пулы серверов
- listen # Frontend + Backend в одном

HTTP mode - для веб-приложений

```
mode http
option httpchk GET /health
balance roundrobin
cookie SERVERID insert
```

TCP mode - для бинарных данных

```
mode tcp
option tcp-check
balance leastconn
option httpchk GET /primary
```

HAProxy

```
defaults
    mode tcp
    timeout connect 3s
    timeout client 1h
    timeout server 1h

# =====
# Redis Master-Slave
# =====
listen redis_master
    bind *:6379
    option tcp-check
    tcp-check send PING\r\n
    tcp-check expect string +PONG
    default-server inter 2s fall 3 rise 2
    server redis1 10.0.4.10:6379 check
    server redis2 10.0.4.11:6379 check backup

# =====
# Redis Read Replicas
# =====
listen redis_replicas
    bind *:6380
    balance leastconn
    server redis2 10.0.4.11:6379 check
    server redis3 10.0.4.12:6379 check
```

ИТМО

/etc/haproxy/haproxy.cfg



- global # Глобальные параметры процесса
- defaults # Значения по умолчанию
- frontend # Точки входа (listeners)
- backend # Пулы серверов
- listen # Frontend + Backend в одном

HTTP mode - для веб-приложений

```
mode http
option httpchk GET /health
balance roundrobin
cookie SERVERID insert
```

TCP mode - для бинарных данных

```
mode tcp
option tcp-check
balance leastconn
option httpchk GET /primary
```

HAProxy

```
defaults
    mode http
    timeout connect 5s
    timeout client 30s
    timeout server 30s

# =====
# Microservices Gateway
# =====
frontend microservices
    bind *:80

    # Маршрутизация по path
    acl is_users path_beg /api/users
    acl is_orders path_beg /api/orders
    acl is_payments path_beg /api/payments

    use_backend users_service if is_users
    use_backend orders_service if is_orders
    use_backend payments_service if is_payments

backend users_service
    balance roundrobin
    server user1 10.0.3.10:8080 check
    server user2 10.0.3.11:8080 check

backend orders_service
    balance leastconn
    server order1 10.0.3.20:8080 check
    server order2 10.0.3.21:8080 check

backend payments_service
    balance roundrobin
    server pay1 10.0.3.30:8080 check
    server pay2 10.0.3.31:8080 check
```

ИТМО

/etc/haproxy/haproxy.cfg



- global # Глобальные параметры процесса
- defaults # Значения по умолчанию
- frontend # Точки входа (listeners)
- backend # Пулы серверов
- listen # Frontend + Backend в одном

HTTP mode - для веб-приложений

```
mode http
option httpchk GET /health
balance roundrobin
cookie SERVERID insert
```

TCP mode - для бинарных данных

```
mode tcp
option tcp-check
balance leastconn
option httpchk GET /primary
```



- ✓ Docker/Kubernetes окружение
- ✓ Нужен zero-config подход
- ✓ Микросервисы с частыми изменениями
- ✗ Не лучший выбор для чистого TCP

- Cloud-native reverse proxy
- Autodiscovery
- Динамическая конфигурация (без перезапуска)

Autodiscovery:

- Docker labels
- Kubernetes Ingress/CRD
- Consul Catalog
- File watching

Let's Encrypt:

- Автоматические SSL сертификаты
- Automatic renewal

Протоколы:

- HTTP/2, HTTP/3
- gRPC
- WebSocket



- Web-сервер + reverse proxy + load balancer
- Асинхронная архитектура (event-driven)
- Популярен для HTTP/HTTPS, но поддерживает TCP/UDP

- ✓ Нужен веб-сервер + балансировщик в одном
- ✓ Больше HTTP, меньше TCP
- ✓ SSL termination с кешированием



ENGINE-X



NNGGJINX

imgflip.com

Designing Data-Intensive Applications

THE BIG IDEAS BEHIND RELIABLE, SCALABLE,
AND MAINTAINABLE SYSTEMS



Martin Kleppmann



Availability - “Дать возможность системе продолжать работать, даже если некоторые части уже не работают” (с)

“Availability означает способность системы маскировать и самовосстанавливаться таким образом, чтобы периоды недоступности сервиса не превышали необходимого с течением какого-то времени”

Что такое High Availability?



- **Availability** подразумевает, что «нечто» может использоваться, как ожидалось (*например: база данных обслуживает транзакции сервиса*)
- **High Availability** подразумевает, что «нечто» не только доступно, но и защищено от состояния недоступности [с помощью специальных инструментов] (*например: база данных дополнительно зеркалируется и будет доступна в случае аварии*)
- Пользователи/сервисы постоянно могут работать с системой
- Что за «нечто»?

Что за «нечто»?



Природа этого «нечто» сильно варьируется, как и упомянутые специальные инструменты

- Таблица: может быть защищена репликацией, или вовсе вся БД
- Группа БД: может быть защищена через availability groups
- Сервер: может быть защищен с помощью кластеризации
- ЦОД: может быть защищен SAN-репликацией
- и т.д.

Не путаем термины



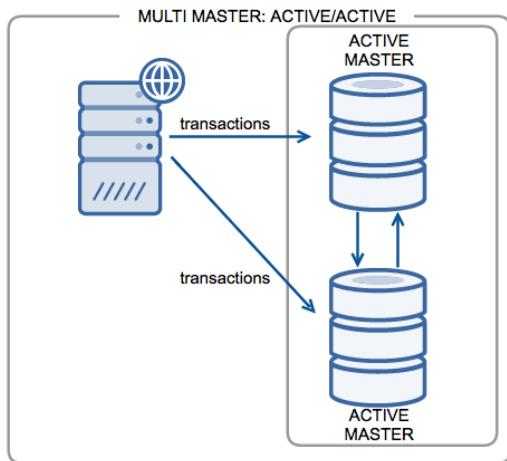
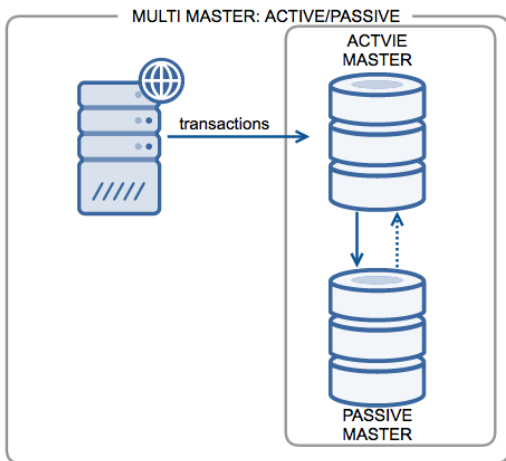
- **Accessibility** – способность *пользователей* получить доступ к системе
- **Availability** – способность *системы* оставаться доступной
- **Fault Tolerance** – способность системы функционировать при отказе компонентов
- **Failover|Switchover** – способность системы минимизировать время простоя благодаря переключению
- **Disaster Recovery** – способность системы восстановиться после сбоя



- ▶ Избыточность на уровне приложений
- ▶ Избыточность на уровне платформ (систем)

- ▶ Задачи:
 - ▶ отказоустойчивость
 - ▶ балансировка
 - ▶ масштабирование

Кластеризация БД



Проблемы одного сервера БД:

- — Single Point of Failure (SPOF)
- — Ограниченная производительность
- — Простои при обновлениях
- — Невозможность горизонтального масштабирования

Основные паттерны:

- **Master-Slave (Primary-Replica)** – удобно для чтения, бутлнек при записи
- **Master-Master (Multi-Master)** – удобно при записи, но опасность split-brain
- **Sharding (горизонтальное разбиение)** – горизонтальное масштабирование, но сложность обработки запросов

Для Postgres

ІТМО



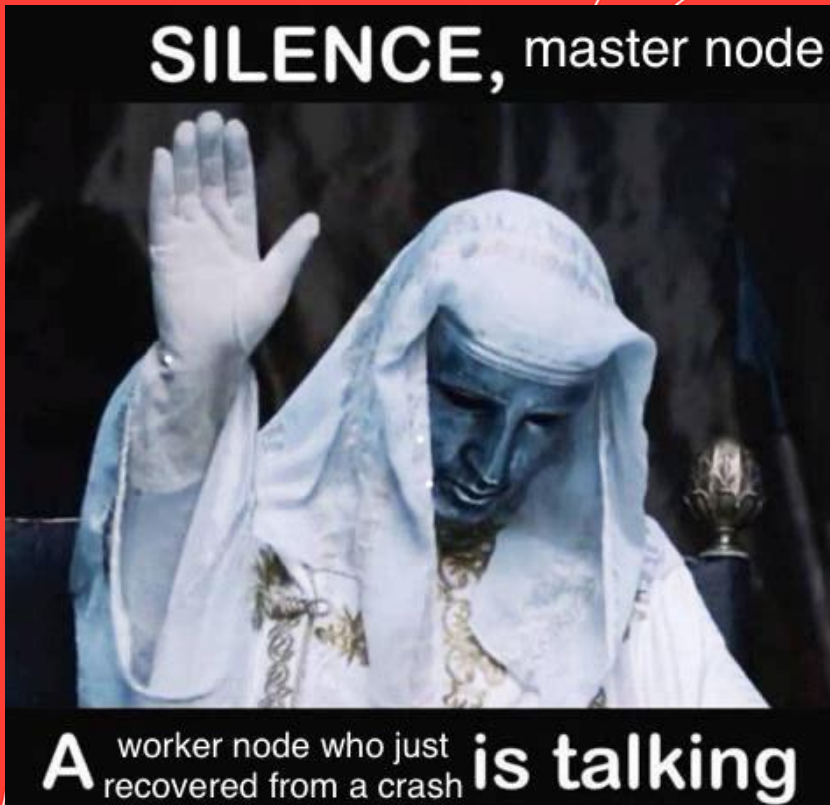
Pgpool-II



PATRONI

STOLON 

+ streaming replication



- **Обнаружение сбоя:** Как понять, что Master действительно упал?
- **Выбор нового Master:** Какую Replica промолчать?
- **Split-brain:** Как избежать двух Master одновременно?
- **Переконфигурация:** Как сообщить приложениям о новом Master?
- **Автоматизация:** Как заметить, но не править руками?

Судьи кто?

ІТМО



- ❌ Дать реплике решать самой -> драка за позицию мастера
- ❌ Добавить одного наблюдателя -> лучше, но он становится точкой отказа
- ✅ Распределенный консенсус -> решает **кворум** из наблюдателей



Distributed Consensus Store (DCS)

Распределенное хранилище данных с гарантией согласованности через консенсус-протокол



Консистентность:

- Все узлы видят одни и те же данные
- Linearizable reads/writes

Доступность с ограничениями:

- Работает при наличии quorum (большинства узлов)
- 3 узла = работает при 1 сбое
- 5 узлов = работает при 2 сбоях

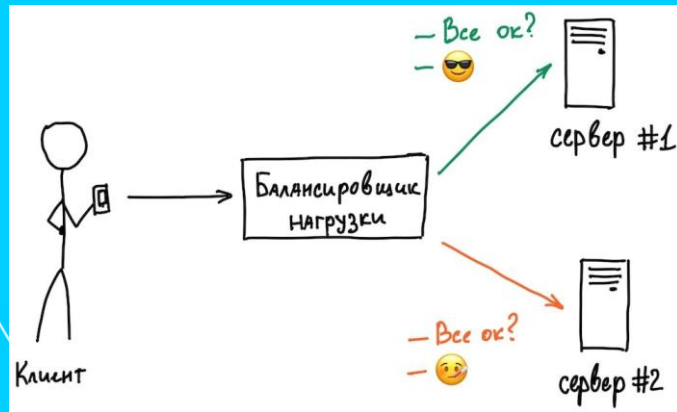
Partition Tolerance:

- При split-brain работает только partition с quorum
- Minority partition становится read-only или недоступна



На примере PostgreSQL Cluster

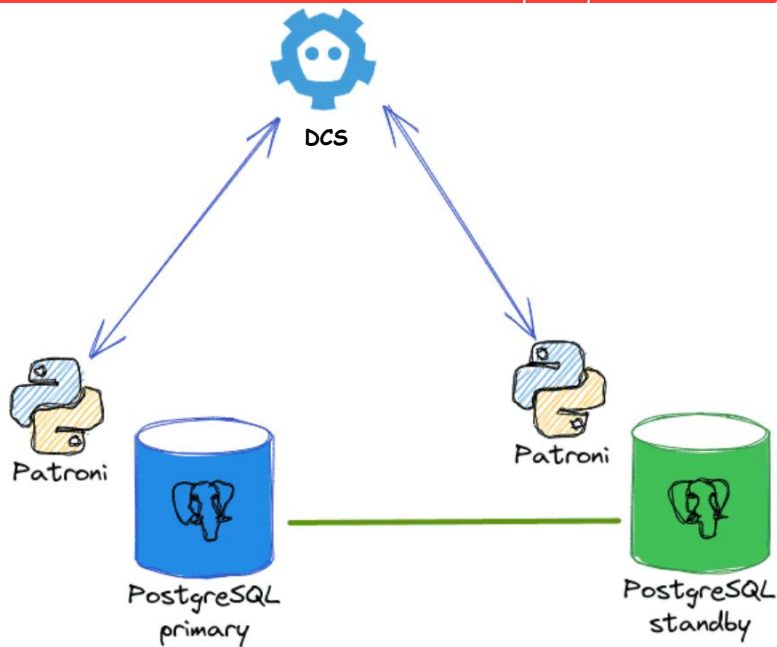
Термин «кластер» в PostgreSQL означает не то же самое, что в понятии «вычислительный кластер». В данном контексте под «кластером» понимается postmaster — серверный процесс, а также набор баз данных под его управлением. Это отличие от вычислительного кластера, объединяющего группу компьютеров для выполнения общих задач



Сущности

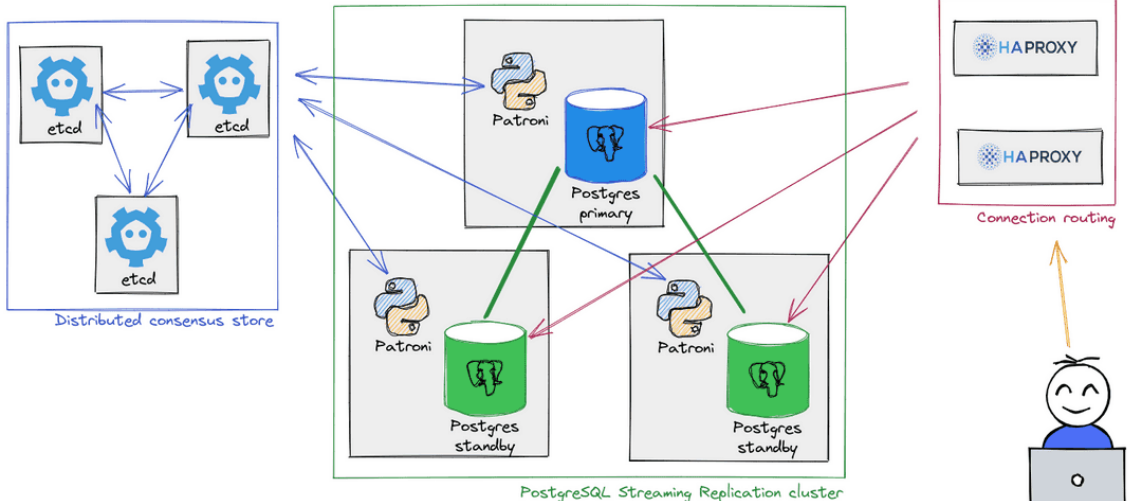
- Сам объект (СУБД PostgreSQL)
- Инструмент кластеризации (Patroni, Stolon, ...)
- Распределенное хранилище конфигураций кластера (etcd, Consul, ZooKeeper, ...)
- Балансировка нагрузки, распределение запросов между нодами кластера (HAProxy, Nginx, ...)
- *Опционально: единая точка входа, шлюз*

На примере PostgreSQL Cluster



- Лидером могут стать только «здоровые» ноды кластера, причем только одна (избегаем сплит-брейн)
- Каждая нода решает только сама за себя
- Состояние кластера хранится в консистентном распределённом хранилище (DCS)
- Ключ лидера изменяется с помощью атомарных операций CAS
- Автоматическая изоляция некорректно работающих или вышедших из строя нод

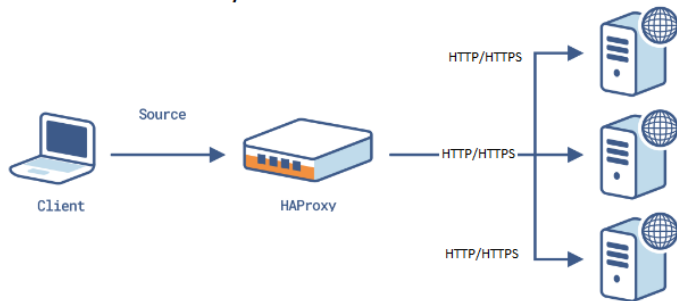
На примере PostgreSQL Cluster



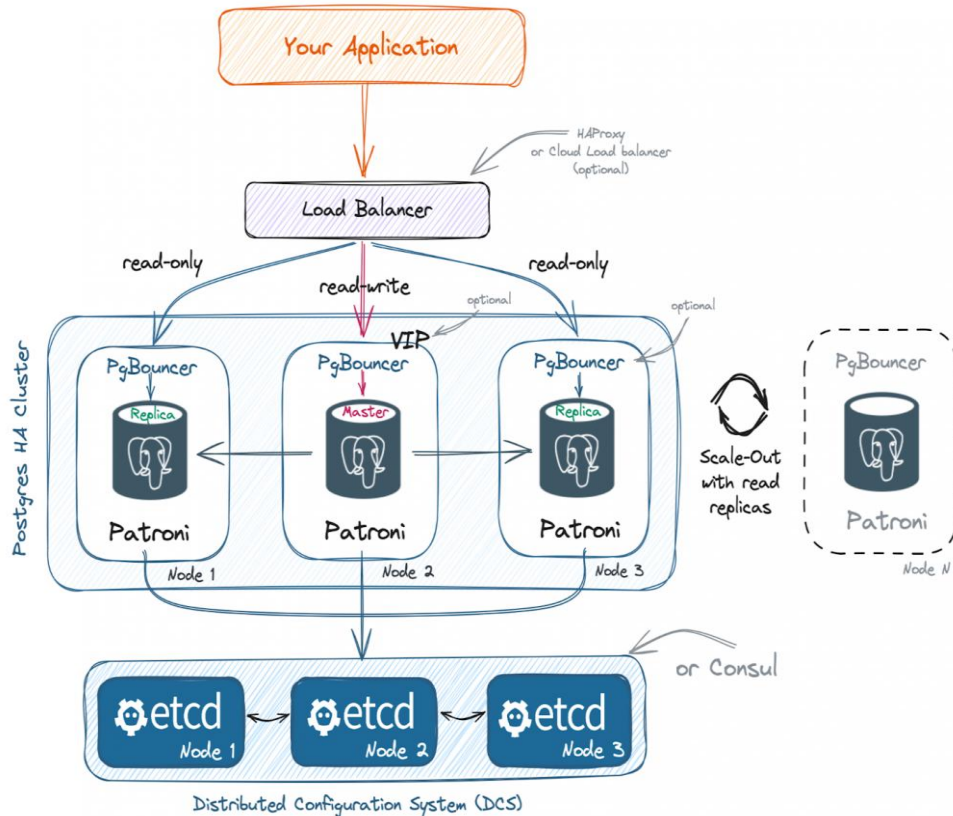
- Лидером могут стать только «здоровые» ноды кластера, причем только одна (избегаем сплит-брейн)
- Каждая нода решает только сама за себя
- Состояние кластера хранится в консистентном распределённом хранилище (DCS)
- Ключ лидера изменяется с помощью атомарных операций CAS
- Автоматическая изоляция некорректно работающих или вышедших из строя нод

На примере PostgreSQL Cluster

HAProxy Load Balancer



Примитивный автофейловер



Термин «кластер» в PostgreSQL означает не то же самое, что в понятии «вычислительный кластер». В данном контексте под «кластером» понимается **postmaster** — серверный процесс, а также набор баз данных под его управлением. Это отличие от вычислительного кластера, объединяющего группу компьютеров для выполнения общих задач

